CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

Practical Aim : To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1 : A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

For Code :

https://github.com/Aarav280907/FDSA_A2_SHP/tree/f6fe6c657d4e94dd100b192902435c73583d8b2e/PRACTICAL-3

Output :

```
PS D:\DSA\FDSA_A2_SHP\PRACTICAL-3\output> & .\'practical3_1.exe'
Bubble Sort: 1 3 5 7 8
Selection Sort: 1 3 5 7 8
Insertion Sort: 1 3 5 7 8
PS D:\DSA\FDSA_A2_SHP\PRACTICAL-3\output>
```

Problem Faced: Facing problem when doing swap and insertion.

Outcome/Conclusion : The teacher's sorting exercise illustrates three fundamental algorithms: bubble sort, selection sort, and insertion sort. Each method successfully arranges the marks in ascending order, but all share the same $O(n^2)$time complexity, making them suitable only for small datasets. The problem highlights how different approaches achieve the same goal with varying strategies—swapping, selecting minimums, or inserting in place. Overall, the conclusion is that while these methods are valuable for learning basic sorting

concepts, more efficient algorithms like merge sort or quicksort are preferred for larger stacks of answer sheets.

Key Question : One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

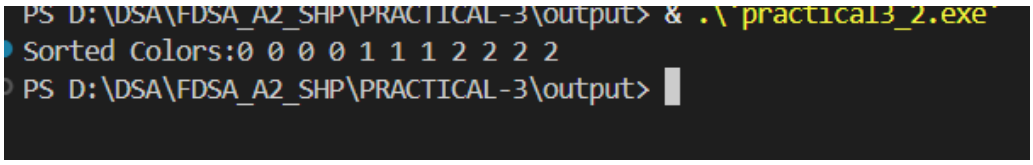
Answer : The method is Bubble Sort. By adding a flag to check if any swaps occur in a pass, it can stop early when the list is already sorted. Selection Sort always scans for the minimum, and Insertion Sort still processes each element, so neither can detect sortedness in advance.

Problem 2 : A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

For Code :

https://github.com/Aarav280907/FDSA_A2_SHP/tree/f6fe6c657d4e94dd100b192902435c73583d8b2e/PRACTICAL-3

Output :



Problem Faced : When try it for first time not able to print 0,1,2 number of times given in input.

Outcome/Conclusion :

By applying the Dutch National Flag algorithm, the paint shop owner can efficiently rearrange the buckets in place with O(n) time and O(1) space, ensuring all 0s, then 1s, then 2s are grouped together.

Key Question : If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

Answer : The second pass happens because the logic first partitions 0s and 2s, then makes another pass to place 1s correctly. That decision to handle 1s separately forces the extra pass. By restructuring the condition so that 0s, 1s, and 2s are all handled in a single traversal (Dutch National Flag approach), you eliminate the need for a second pass.